Java Programming Training

Day 2

Complete Program Reference

17 Programs · Full Theory · Explanations · Trace Tables

SECTION A — For Loops

Iteration, tables, and printing sequences using the for loop

■ Theory — What is a for Loop?

A for loop is a control flow statement used when you know exactly how many times you need to repeat a block of code. Unlike a while loop (which only has a condition), a for loop packs three things into one compact line: initialization (sets a counter), condition (when to stop), and update (how to advance). This makes for loops the preferred choice for counting tasks, table generation, and iterating over ranges.

Part	Syntax Example	What It Does	When It Runs
Initialization	int i = 0	Creates the loop counter variable	Once, at the very start
Condition	i < 10	Loop continues while this is true	Before every iteration
Update	i++	Advances the counter	After every iteration

PROGRAM 01

ForLoop.java

Print even numbers 0–9 using for loop + modulo

■ CONCEPT — Modulo Operator (%) + Conditional Filtering

The modulo operator % gives the REMAINDER after division.

chinki % 2 == 0 → 'when divided by 2, no remainder' → the number is EVEN.

Odd numbers (1, 3, 5…) give remainder 1, so the if condition is false and they are skipped.

This pattern — loop + modulo filter — appears everywhere: fizzbuzz, digit extraction, cycle detection.

■ ForLoop.java

```
package Day2;
public class ForLoop {
public static void main(String[] args) {
// 'chinki' is just a counter variable — could be 'i', 'x', anything
for (int chinki = 0; chinki < 10; chinki++) {
// % 2 == 0 → even number check
if (chinki % 2 == 0) {
System.out.println(chinki);
}
}
}
}
```

■ Output

```
0
```

```
2
4
6
8
```

Iteration (chinki)	chinki % 2	Condition == 0?	Printed?
0	0	■ Yes	0
1	1	■ No	—
2	0	■ Yes	2
3	1	■ No	—
4	0	■ Yes	4
...	...	...	...
9	1	■ No	—
10	—	Loop ends	—

■ TIP

Variable names can be anything — here 'chinki' is used instead of the typical 'i'. Java only cares about the logic, not the name. Use meaningful names in real projects.

PROGRAM 02

Table.java

Print multiplication table using Scanner + for loop

■ CONCEPT — Scanner Class — Reading User Input

Scanner is a Java class (in java.util package) that reads input from the keyboard (System.in).

sc.nextInt() pauses execution and waits for the user to type a number and press Enter.

You MUST import java.util.Scanner at the top — without the import, Java doesn't know what Scanner is.

This program shows the most common pattern: read once with Scanner, then loop to use the value.

■ Table.java

```
package Day2;
import java.util.Scanner;
public class Table {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in); // Create scanner object
System.out.print("Enter number: ");
int num = sc.nextInt(); // Read user input
// Loop from 1 to 10 to print multiplication table
for (int i = 1; i <= 10; i++) {
```

```
System.out.println(num + " x " + i + " = " + (num * i));
}
}
}
```

■ Output

```
Enter number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
... (up to)
5 x 10 = 50
```

i value	Calculation	Output line
1	5 × 1 = 5	5 x 1 = 5
2	5 × 2 = 10	5 x 2 = 10
5	5 × 5 = 25	5 x 5 = 25
10	5 × 10 = 50	5 x 10 = 50
11	i <= 10 is false	Loop exits

■ WHY THIS WORKS

Why (num * i) in parentheses? Without parentheses, Java concatenates: '5 x 3 = ' + num + '*' + i would print '5 x 3 = 53' (string join). Parentheses force arithmetic to happen first.

SECTION B — While Loops & Number Logic

Digit extraction, counting, reverse, Armstrong, Palindrome, Prime, Fibonacci, Banking App

■ Theory — The while Loop & Digit Extraction Pattern

A while loop repeats as long as a condition is true. It's preferred over for when you don't know how many iterations are needed upfront — like processing digits of an unknown number. The two foundational digit operations that power 7 of the programs in this section are: • digit = number % 10 — extracts the LAST digit (remainder when divided by 10) • number = number / 10 — REMOVES the last digit (integer division discards the remainder) By repeating these two lines until number == 0, you process every digit one at a time.

Pattern	Operation	Example (num=153)	Result
Extract last digit	num % 10	153 % 10	3
Remove last digit	num / 10	153 / 10	15
Extract last digit	num % 10	15 % 10	5
Remove last digit	num / 10	15 / 10	1
Extract last digit	num % 10	1 % 10	1
Remove last digit	num / 10	1 / 10	0 → loop ends

PROGRAM 03

CountNumber.java

Count digits in a number using while loop

■ CONCEPT — Digit Counting — Why /10 works

Each time we do number / 10 (integer division), we chop off the last digit.

Counting how many chops until number == 0 = counting how many digits there were.

This logic is reused inside ArmstrongNumber.java to determine the power to raise each digit to.

```
package Day2;
public class CountNumber {
public static void main(String[] args) {
int input = 45217890;
int count = 0;
while (input != 0) {
count++; // Count this digit
input = input / 10; // Chop last digit off
}
System.out.println("Total no of digit were : " + count);
}
}
```

■ Output

```
Total no of digit were : 8
```

Iteration	input (before)	input / 10 (after)	count
1	45217890	4521789	1
2	4521789	452178	2
3	452178	45217	3
4	45217	4521	4
5	4521	452	5
6	452	45	6
7	45	4	7
8	4	0	8
Exit	0	Loop ends	8

PROGRAM 04

GreatestNumber.java

Find the greatest digit in a number

■ CONCEPT — Running Maximum — Comparing while Extracting

Initialize greatest = 0 (the smallest possible digit value).

Extract each digit with % 10, compare to current greatest, update if larger.

Remove the digit with / 10 and repeat. At the end, greatest holds the largest digit found.

```
package Day2;
public class GreatestNumber {
public static void main(String[] args) {
int input = 123456789;
int greatest = 0; // 0 is safe: all digits are 0–9
while (input != 0) {
int digit = input % 10; // Extract last digit
if (digit > greatest) { // Is it bigger than current best?
greatest = digit; // Update the winner
}
input = input / 10; // Remove last digit
}
System.out.println("Greatest digit is: " + greatest);
}
}
```

■ Output

```
Greatest digit is: 9
```

■ WHY THIS WORKS

Why greatest = 0? Digits are 0–9, so starting at 0 guarantees any non-zero digit will trigger an update. If you started at 9, you'd miss smaller digits. If all digits are 0, result stays 0 — correct.

PROGRAM 05

ArmstrongNumber.java

Check Armstrong number using Math.pow()

■ CONCEPT — Armstrong Number + Math.pow() — Two-Pass Algorithm

An Armstrong (narcissistic) number equals the SUM of its digits each raised to the power of the total number of digits. Example: 153 → $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ ✓

Algorithm: PASS 1 — count the digits. PASS 2 — extract each digit, raise to that count using Math.pow(digit, count), and accumulate. Compare final sum to original number.

Math.pow(a, b) returns a^b as a double — so sum must be declared as double.

```
package Day2;
public class ArmstrongNumber {
public static void main(String[] args) {
int num = 153;
int temp = num;
int count = 0;
// PASS 1: Count digits
while (temp != 0) {
count++;
temp = temp / 10;
}
// PASS 2: Sum of digits^count
temp = num; // Reset temp — reuse for digit extraction
double sum = 0; // double because Math.pow() returns double
while (temp != 0) {
int digit = temp % 10;
sum = sum + Math.pow(digit, count); // digit raised to power count
temp = temp / 10;
}
if (sum == num) {
System.out.println("Its a armstrong");
}
}
}
```

■ Output

```
Its a armstrong
```

Digit Extracted	Math.pow(digit, 3)	Running sum
3 (from 153)	$3^3 = 27$	27

5 (from 15)	$5^3 = 125$	152
1 (from 1)	$1^3 = 1$	153
0 (loop ends)	—	153 == 153 ■

Feature	Day 2 Version (this)	Day 1 Version
Power method	Math.pow(digit, count)	digit * digit * digit
Works for	Any n-digit number	Only 3-digit numbers
sum type	double (Math.pow returns double)	int
Other Armstrong #s	0, 1, 153, 370, 371, 407	153 only

PROGRAM 06

Palindrome.java

Check if a number reads the same forwards and backwards

■ CONCEPT — Number Reversal — The Core Formula

To reverse a number, we build it up digit by digit from the back.

The key formula is: reverse = reverse * 10 + rem

• Multiplying reverse by 10 SHIFTS all existing digits one place LEFT (makes room on the right).

• Adding rem PLACES the new digit on the rightmost position.

After extracting all digits, compare the reversed number with the saved original (temp).

```
package Day2;
public class Palindrome {
public static void main(String[] args) {
int num = 458;
int temp = num; // Save original BEFORE we modify num
int reverse = 0;
while (num != 0) {
int rem = num % 10; // Extract last digit
reverse = reverse * 10 + rem; // Shift left + append digit
num = num / 10; // Remove last digit
}
if (temp == reverse) {
System.out.println("Palindrome");
} else {
System.out.println("Not a Palindrome");
}
}
}
```

■ Output

```
Not a Palindrome (because 458 reversed = 854)
```

num	rem = num%10	reverse = reverse*10 + rem	num / 10
121	1	0×10 + 1 = 1	12
12	2	1×10 + 2 = 12	1
1	1	12×10 + 1 = 121	0
0	—	Loop ends: reverse = 121	—
Compare	121 == 121	■ PALINDROME	

PROGRAM 07

PrimeNumber.java

Check if a number is prime using flag variable

■ CONCEPT — Prime Numbers & Flag Variable Pattern

A prime number has exactly 2 divisors: 1 and itself (2, 3, 5, 7, 11, 13 ...).

A flag variable is a counter/boolean that tracks a condition across a loop.

Here, flag counts how many divisors num has. If flag == 2 at the end → prime.

The original code has a bug: the 'break' exits after the first divisor (always 1),

so flag never reaches 2. Fix: REMOVE the break — let the loop count ALL divisors.

```
package Day2;
import java.util.Scanner;
public class PrimeNumber {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter number: ");
int num = sc.nextInt();
int flag = 0; // Counts number of divisors found
for (int i = 1; i <= num; i++) {
if (num % i == 0) { // i divides num with no remainder?
flag++; // i is a divisor → increment count
// ■ 'break;' was here in original — that was the bug!
// Removing break lets the loop find ALL divisors
}
}
if (flag == 2) { // Exactly 2 divisors = prime
System.out.println(num + " is Prime Number");
} else {
System.out.println(num + " is Not Prime Number");
}
}
}
```

■ Output

```
Enter number: 7
7 is Prime Number

Enter number: 8
8 is Not Prime Number
```

■ BUG / FIX

BUG in original code: The break statement exits the loop after finding divisor 1 (always the first divisor). So flag is always 1, and flag == 2 is never true — every number shows 'Not Prime'. Fix: delete the break line. The corrected version above works correctly.

PROGRAM 08

FibonacciSeries.java

Generate Fibonacci series using variable shift

■ CONCEPT — Fibonacci Sequence — Variable Shift Logic

Each Fibonacci number is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8, 13, 21...

Three variables are used: a (previous), b (current), c (next = a + b).

After computing c, we SHIFT: a becomes b, b becomes c — slide the window forward.

a=0, b=1 are printed first (hardcoded), then the loop generates length more terms.

```
package Day2;
public class FibonacciSeries {
public static void main(String[] args) {
int length = 7; // Number of additional terms after the first two
int a = 0;
int b = 1;
System.out.print(a + " " + b); // Print first two terms: 0 1
for (int i = 1; i <= length; i++) {
int c = a + b; // Next Fibonacci number
System.out.print(" " + c + " ");
a = b; // Shift: a moves forward
b = c; // Shift: b moves forward (new 'current')
}
}
}
```

■ Output

```
0 1 1 2 3 5 8 13 21
```

i	a	b	c = a+b	Printed	Shift: a=b, b=c
(start)	0	1	—	0 1	—
1	0	1	1	1	a=1, b=1

2	1	1	2	2	a=1, b=2
3	1	2	3	3	a=2, b=3
4	2	3	5	5	a=3, b=5
5	3	5	8	8	a=5, b=8
6	5	8	13	13	a=8, b=13
7	8	13	21	21	a=13, b=21

PROGRAM 09

BankWhileLoop.java

Full banking menu app — while + switch + Scanner

■ CONCEPT — Menu-Driven Program — while + switch + Scanner Combined

This is your most complete Day 2 program. It combines three structures:

1. while loop — keeps the menu running until the user selects Exit (option 5)

2. switch statement — routes each option to the right block of code

3. Scanner — reads the user's choice and transaction amounts

This pattern (while + switch) is the standard structure for ALL menu-driven applications.

Option	Action	Logic Used	Key Line
1 — Withdraw	Deduct amount if sufficient	if-else inside case	balance -= amt
2 — Deposit	Add amount to balance	Compound assignment	balance += amtToDeposit
3 — Balance	Display current balance	Simple print	println(balance)
4 — FD	Calculate after Fixed Deposit	SI formula	(balance*7*time)/100
5 — Exit	Break the while loop	while(option!=5)	loop condition false
default	Invalid input message	default: in switch	Abe andha hai kyaaa!!!

```
package Day2;
import java.util.Scanner;
public class BankWhileLoop {
public static void main(String[] args) {
int balance = 2000;
Scanner obj = new Scanner(System.in);
int option = 0;
while (option != 5) { // Keep looping until user chooses Exit
System.out.println("\n1.Withdrawl 2.Deposit 3.CurrentBalance 4.FD 5.Exit");
option = obj.nextInt();
switch (option) {
case 1:
System.out.println("Enter the amt to withdraw");
int amt = obj.nextInt();
if (balance >= amt) {
balance -= amt;
System.out.println(amt + " is debited from your account");
} else {
System.out.println("Insufficient Balance");
}
break;
case 2:
System.out.println("Enter the amt to deposit");
int amtToDeposit = obj.nextInt();
balance += amtToDeposit;
System.out.println("Amount Deposited Successfully");
break;
```

```
case 3:
System.out.println("Current Balance = " + balance);
break;
case 4:
System.out.println("Enter the time to deposit your money");
int time = obj.nextInt();
System.out.println("Your amt after " + time + " years will be "
+ (balance + (balance * 7 * time) / 100));
break;
case 5:
System.out.println("Thank You!");
break;
default:
System.out.println("Abe andha hai kyaaa!!!");
break;
}
}
}
}
```

■ **Output**

```
1.Withdrawl 2.Deposit 3.CurrentBalance 4.FD 5.Exit

3

Current Balance = 2000


1.Withdrawl 2.Deposit 3.CurrentBalance 4.FD 5.Exit

1

Enter the amt to withdraw

500

500 is debited from your account


1.Withdrawl 2.Deposit 3.CurrentBalance 4.FD 5.Exit

5

Thank You!
```

■ **WHY THIS WORKS**

FD Formula: balance + (balance × 7 × time) / 100. This is Simple Interest (SI = P×R×N/100) at 7% added to the principal. Integer division is used — for exact decimal results, cast to double: (double)(balance * 7 * time) / 100.

SECTION C — Pattern Programs

Nested for loops — pyramids, diamonds, hollow squares, and Pascal's Triangle

■ Theory — Nested Loops for Patterns

All 8 pattern programs use NESTED for loops: a loop inside a loop. The OUTER loop controls which ROW you're on (iterates top to bottom). The INNER loop(s) control what gets PRINTED on each row (stars and/or spaces). The relationship between the outer loop variable (row) and inner loop limits (how many spaces, how many stars) is the entire secret to every pattern.

Inner Loop Purpose	What it Prints	Loop Condition Example	Used in
Print stars	* characters	col <= rows	LeftPyramid, ReverseLeftPyramid
Print leading spaces	blank spaces before stars	space <= line - row	RightPyramid, Rhombus
Print fixed stars	same count every row	star <= line	RhombusPattern
Boundary check	* on edges, space inside	if row==1||col==1...	HollowSquare
Upper + lower half	two separate loops	row: 1→N then N→1	DiamondPattern

PROGRAM 10

LeftPyramid.java

Left-aligned — rows of increasing stars

■ CONCEPT — Nested Loop Logic

Outer loop rows: 1→5. Inner loop col: 1→rows. When rows=3, col runs 1,2,3 → 3 stars. Each row prints exactly as many stars as the row number.

```
for (int rows = 1; rows <= 5; rows++) {
for (int col = 1; col <= rows; col++) {
System.out.print("*");
}
System.out.println();
}
```

■ Output

```
*
**
***
****
*****
```

Row	col range	Stars
1	1 to 1	*

2	1 to 2	**
3	1 to 3	***
4	1 to 4	****
5	1 to 5	*****

PROGRAM 11

RightPyramid.java

Right-aligned pyramid — spaces push stars to the right

■ CONCEPT — Nested Loop Logic

Two inner loops per row: first prints (line-row) spaces to push stars right, then prints row stars. As row increases, spaces decrease and stars increase.

```
int line = 4;
for (int row = 1; row <= line; row++) {
for (int space = 1; space <= line - row; space++) {
System.out.print(" ");
}
for (int star = 1; star <= row; star++) {
System.out.print("*");
}
System.out.println();
}
```

■ Output

```
*
**
***
****
```

Row	Spaces (line-row)	Stars (row)	Line
1	3	1	*
2	2	2	**
3	1	3	***
4	0	4	****

PROGRAM 12

ReverseLeftPyramid.java

Decreasing left pyramid — outer loop counts down

■ CONCEPT — Nested Loop Logic

Only one change from LeftPyramid: outer loop starts at 5 and counts DOWN (rows--). Row 1 prints 5 stars, row 2 prints 4, etc. Inner loop unchanged.

```
for (int rows = 5; rows >= 1; rows--) {
for (int col = 1; col <= rows; col++) {
System.out.print("*");
}
System.out.println();
}
```

■ Output

```
*****

****

***

**

*
```

Row (rows value)	Stars printed	Line
5	5	*****
4	4	****
3	3	***
2	2	**
1	1	*

PROGRAM 13

ReverseRightPyramid.java

Shrinking right-aligned — spaces increase as stars decrease

■ CONCEPT — Nested Loop Logic

Outer loop counts DOWN. Space formula (line-row) now INCREASES as row decreases, pushing smaller star groups further right. Stars decrease each row.

```
int line = 4;
for (int row = line; row >= 1; row--) {
for (int space = 1; space <= line - row; space++) {
System.out.print(" ");
}
for (int star = 1; star <= row; star++) {
System.out.print("*");
```

```
}
System.out.println();
}
```

■ Output

```
****

***

**

*
```

row	Spaces (line-row)	Stars	Line
4	0	4	****
3	1	3	***
2	2	2	**
1	3	1	*

PROGRAM 14

RhombusPattern.java

Rhombus — fixed star count, sliding space

■ CONCEPT — Rhombus — What Makes it Different from Pyramids

In ALL pyramids, the star count changes per row. In a Rhombus, star count is FIXED (= line every row).

Only the leading SPACES change — they decrease from (line-1) down to 0.

This causes the block of stars to slide left row by row, creating the rhombus/parallelogram shape.

```
int line = 5;
for (int row = 1; row <= line; row++) {
for (int space = 1; space <= line - row; space++) {
System.out.print(" "); // Spaces decrease each row
}
for (int star = 1; star <= line; star++) {
System.out.print("*"); // Stars: ALWAYS line (fixed = 5)
}
System.out.println();
}
```

■ Output

```
*****

*****

*****

*****

*****
```

Row	Spaces (line-row)	Stars (fixed=line)	Output
1	4	5	*****
2	3	5	*****
3	2	5	*****
4	1	5	*****
5	0	5	*****

PROGRAM 15

HollowSquare.java

*Hollow square — print * only on boundary cells*

■ CONCEPT — Boundary Condition — if/else Inside Nested Loop

A full square always prints *. A HOLLOW square only prints * on the 4 edges.

The boundary condition checks: is this cell on the first row, last row, first col, or last col?

if (row==1 || row==line || col==1 || col==line) → print * else → print space

This OR-chained boundary check is a classic pattern used in grid, matrix, and board problems.

```
int line = 4;
for (int row = 1; row <= line; row++) {
for (int col = 1; col <= line; col++) {
// Print * only on the 4 edges of the square
if (row == 1 || row == line || col == 1 || col == line) {
System.out.print("*");
} else {
System.out.print(" "); // Interior = hollow space
}
}
System.out.println();
}
```

■ Output

```
****

* *

* *

****
```

Cell (row, col)	Condition true?	Reason	Print
(1, 1)	■	row==1 (top edge)	*
(1, 4)	■	row==1 (top edge)	*
(2, 1)	■	col==1 (left edge)	*

(2, 2)	■	Interior cell	(space)
(2, 3)	■	Interior cell	(space)
(2, 4)	■	col==line (right edge)	*
(4, 2)	■	row==line (bottom edge)	*

PROGRAM 16

DiamondPattern.java

Diamond — two separate loops for upper and lower halves

■ CONCEPT — Two-Part Pattern — Upper Expanding + Lower Shrinking

A diamond cannot be made with one loop. It requires TWO separate nested loops:

UPPER HALF (rows 1→line): spaces decrease, stars increase row by row

LOWER HALF (rows line-1→1): spaces increase, stars decrease row by row

Stars formula: (row - 1) spaced pairs → each star printed with a space after it

Spaces formula: (line - row) leading spaces

```
int line = 5;
// ■■■ UPPER HALF (expanding) ■■■
for (int row = 1; row <= line; row++) {
for (int space = 1; space <= line - row; space++) {
System.out.print(" ");
}
for (int star = 1; star <= (row - 1); star++) {
System.out.print("* ");
}
System.out.println();
}
// ■■■ LOWER HALF (shrinking) ■■■
for (int row = line - 1; row >= 1; row--) {
for (int space = 1; space <= line - row; space++) {
System.out.print(" ");
}
for (int star = 1; star <= (row - 1); star++) {
System.out.print("* ");
}
System.out.println();
}
```

■ Output

```
*
* *
* * *
* * * *
* * *
* *
*
```

Half	row range	Stars (row-1)	Spaces (line-row)
Upper	1 → line	0, 1, 2, 3, 4	4, 3, 2, 1, 0
Lower	line-1 → 1	3, 2, 1, 0	1, 2, 3, 4

PROGRAM 17

PascalsTrianglePattern.java

Centered star pyramid — visual shape of Pascal's Triangle

■ CONCEPT — Centered Triangle with '* ' Spacing

This prints the VISUAL LAYOUT of Pascal's Triangle (not the actual numbers).

Each row has (line-row) leading spaces for centering + row stars each printed as '* ' (star+space).

The '* ' spacing is the key visual difference from RightPyramid — it gives wider, more distinct rows.

Looks like: top-center peak expanding downward in a triangle shape.

```
int line = 4;
for (int row = 1; row <= line; row++) {
// Leading spaces for centering
for (int space = 1; space <= line - row; space++) {
System.out.print(" ");
}
// Stars with space after each one
for (int star = 1; star <= row; star++) {
System.out.print("* ");
}
System.out.println();
}
```

■ Output

```
*

* *

* * *

* * * *
```

Row	Spaces (line-row)	Stars (row)	Output line
1	3	1	*
2	2	2	* *
3	1	3	* * *
4	0	4	* * * *

■ TIP

Compare to RightPyramid: Both have the same space and star formulas. The only difference is '* ' (star+space) here vs just '*' in RightPyramid. That extra space gives the wider, centered look resembling Pascal's Triangle rows.

SECTION D — Master Quick Reference

All 17 programs at a glance — concepts, key lines, and pattern formulas

All 17 Programs — Complete Reference

#	Program	Core Concept	Key Line / Formula	Output
01	ForLoop.java	for loop + modulo	chinki % 2 == 0	0 2 4 6 8
02	Table.java	for + Scanner	num * i	5 x 3 = 15
03	CountNumber.java	while + /10	count++; input/=10	8 digits
04	GreatestNumber.java	while + if + %	greatest = digit	9
05	ArmstrongNumber.java	while + Math.pow	sum+=Math.pow(d,count)	Armstrong ✓
06	Palindrome.java	while + reversal	rev=rev*10+rem	Palindrome/Not
07	PrimeNumber.java	for + flag (no break!)	flag++ → flag==2	Prime/Not
08	FibonacciSeries.java	for + var shift	c=a+b; a=b; b=c	0 1 1 2 3 5...
09	BankWhileLoop.java	while+switch+Scanner	balance -= amt	Full menu app
10	LeftPyramid.java	nested for	col <= rows	* ** ***
11	RightPyramid.java	nested + spaces	space <= line-row	right-aligned
12	ReverseLeftPyramid.java	nested (rows--)	rows = 5 → 1	***** **** ...
13	ReverseRightPyramid.java	nested + spaces	space <= line-row	shrink+shift
14	RhombusPattern.java	nested + fixed stars	star <= line (fixed)	sliding block
15	HollowSquare.java	nested + boundary if	row==1||row==N||...	**** * * ****
16	DiamondPattern.java	2 nested loops	upper+lower half	◆ shape
17	PascalsTrianglePattern.java	nested '* ' print	star <= row	▲ shape

Pattern Programs — Nested Loop Formula Cheat Sheet

Pattern	Outer Loop	Inner Loop 1 (Spaces)	Inner Loop 2 (Stars)
LeftPyramid	row: 1 → N	—	col: 1 → row
RightPyramid	row: 1 → N	space: 1 → (N-row)	star: 1 → row
ReverseLeftPyramid	row: N → 1	—	col: 1 → row
ReverseRightPyramid	row: N → 1	space: 1 → (N-row)	star: 1 → row
RhombusPattern	row: 1 → N	space: 1 → (N-row)	star: 1 → N (FIXED)
HollowSquare	row: 1 → N	col: 1 → N	if boundary → * else space
DiamondPattern	row: 1→N then N→1	space: 1 → (N-row)	star: 1 → (row-1)

PascalsTriangle	row: 1 → N	space: 1 → (N-row)	star: 1 → row (print '* ')

Key Java Concepts Used — Theory Recap

Concept	What It Is	Used In
% (modulo)	Remainder after division. n%2==0 → even	ForLoop, Prime, Armstrong, Palindrome
/ (integer div)	Divides and DROPS the decimal. 153/10=15	CountNumber, ArmstrongNumber, Palindrome
Math.pow(a,b)	Returns a^b as double. In java.lang (auto-import)	ArmstrongNumber
Scanner	Reads keyboard input. Needs import java.util.Scanner	Table, PrimeNumber, BankWhileLoop
while loop	Repeats while condition is true. Good for unknown count	Programs 03–09
for loop	Repeats a fixed number of times. counter in header	Programs 01–02, 07–08, 10–17
switch-case	Routes to matching case. Needs break to avoid fallthrough	BankWhileLoop
Nested loops	Loop inside a loop. Outer=rows, Inner=cols/stars/spaces	All patterns 10–17
Flag variable	Counter/bool tracking a condition across a loop	PrimeNumber
Boundary check	OR condition on edges: row==1||row==N||col==1||col==N	HollowSquare

Java Day 2 Training · Anurag Bodkhe · MIT Art, Design and Technology University · 17 Programs · For Loops · While Loops · Patterns · Switch · Scanner